

ECE 60146 HW 04

Andrea Goh (goh8@purdue.edu)

February 2025

1 Introduction

The goal of this assignment is to assess how classification accuracy is influenced by object quantity, diversity, and whether the object is the primary focus of the image. To do so, we begin by creating three different dataset variants. The same CNN model will be trained on each of the datasets variants and classification accuracy for each will be analyzed. The three datasets created will be as follows:

1. Single-instance dataset: Images containing only one instance of an object.
2. Multi-instance (same object) dataset: Images containing multiple instances of the same object.
3. Multi-instance (different objects) dataset: Images containing multiple objects from different categories.

2 Programming Tasks

2.1 Create Image Classification Dataset

```
1
2 from pycocotools.coco import COCO
3 import os
4 from PIL import Image
5 import sys
6 import matplotlib.pyplot as plt
7 from torchvision import transforms
8 from tabulate import tabulate
9
10 # Set COCO dataset paths
11 img_dir = 'train2014'
12 ann_file = 'annotations_trainval2014/annotations/instances_train2014.json'
13 output_dir_all = 'output_datasets_all'
14 output_dir_suitable = 'output_dataset'
15
16 # Load COCO dataset
17 coco = COCO(ann_file)
18
19 # Ensure output directories exist
20 os.makedirs(output_dir_all, exist_ok=True)
21 os.makedirs(output_dir_suitable, exist_ok=True)
22
23 def extract_imgs():
24     cat_ids = coco.getCatIds()
25     categories = coco.loadCats(cat_ids)
26     cat_names = [cat['name'] for cat in categories]
27     for cat_name in cat_names:
28         cat_ids = coco.getCatIds(catNms=cat_name)
29         img_ids = coco.getImgIds(catIds=cat_ids)
30
31         for img_id in img_ids:
32             img_info = coco.loadImgs(img_id)[0]
33             ann_ids = coco.getAnnIds(imgIds=img_id, iscrowd=False)
34             anns = coco.loadAnns(ann_ids)
35
36             obj_counts = {} # Initialize dictionary to keep track of object categories
37                             and count
```

```

37
38     for ann in anns:
39         obj_category = coco.loadCats(ann['category_id'])[0]['name']
40         obj_counts[obj_category] = obj_counts.get(obj_category,0) + 1
41
42     if len(obj_counts) >= 2: # if multiple object categories are present
43         save_image(img_info, cat_name, 'multi_instance_different_dataset')
44
45     else:
46         if obj_counts[obj_category] == 1:
47             save_image(img_info, cat_name, 'single_instance_dataset')
48         else:
49             save_image(img_info, cat_name, 'multi_instance_same_dataset')
50
51 def save_image(img_info, category_name, dataset_type):
52     img_path = os.path.join(img_dir, img_info['file_name'])
53     save_dir = os.path.join(output_dir_all, dataset_type, category_name)
54     os.makedirs(save_dir, exist_ok=True)
55
56     # Load and resize image
57     img = Image.open(img_path).resize((64, 64))
58     img.save(os.path.join(save_dir, img_info['file_name']))
59
60 def check_suitable_dataset():
61     for dataset_type in os.listdir(output_dir_all):
62         print(f'Suitable for {dataset_type}')
63         for cat in os.listdir(os.path.join(output_dir_all, dataset_type)):
64             files = [f for f in os.listdir(os.path.join(output_dir_all, dataset_type,
65                                                             cat))]
66             if len(files) >= 500:
67                 print(f'{cat}: {len(files)} (Without augmentation)')
68             elif len(files) >= 350:
69                 print(f'{cat}: {len(files)} (With < 150 augmentations)')
70             elif len(files) >= 300:
71                 print(f'{cat}: {len(files)} (With 150 - 200 augmentations)')
72             elif len(files) >= 250:
73                 print(f'{cat}: {len(files)} (With 200-250 augmentations)')
74
75     return
76
77 def gen_suitable_dataset(suitable_cat):
78     for dataset_type in os.listdir(output_dir_all):
79         for cat in suitable_cat:
80             files = [f for f in os.listdir(os.path.join(output_dir_all, dataset_type,
81                                                             cat))]
82             num_training = min(400, int(0.8 * len(files)))
83             num_total = min(500, len(files))
84             save_dir_train = os.path.join(output_dir_suitable, dataset_type, cat, '
85                                     training')
86             save_dir_validation = os.path.join(output_dir_suitable, dataset_type, cat, '
87                                     validation')
88             for file in files[:num_training]:
89                 img = Image.open(os.path.join(output_dir_all, dataset_type, cat, file))
90                 os.makedirs(save_dir_train, exist_ok=True)
91                 img.save(os.path.join(save_dir_train, file))
92             for file in files[num_training:num_total]:
93                 img = Image.open(os.path.join(output_dir_all, dataset_type, cat, file))
94                 os.makedirs(save_dir_validation, exist_ok=True)
95                 img.save(os.path.join(save_dir_validation, file))
96
97             if num_total < 500:
98                 transform_custom = transforms.Compose([
99                     transforms.RandomHorizontalFlip(),
100                     transforms.RandomRotation(20),
101                     transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2,
102                                             hue=0.2),
103                 ])
104                 train_name = int(files[num_training-1][15:27]) + 1
105                 validation_name = int(files[-1][15:27]) + 1
106                 for i in range(400-int(0.8*(num_total))):
107                     img = Image.open(os.path.join(output_dir_all, dataset_type, cat,
108                                                     files[i%num_total]))
109                     img_transformed = transform_custom(img)

```

```

103         img_transformed.save(os.path.join(save_dir_train, f'COCO_train2014_{
104             train_name}.jpg'))
105         train_name += 1
106         for i in range(100-int(0.2*num_total)):
107             img = Image.open(os.path.join(output_dir_all, dataset_type, cat,
108                 files[len(files)-(i%(num_total-num_training))-1]))
109             img_transformed = transform_custom(img)
110             img_transformed.save(os.path.join(save_dir_validation, f'
111                 COCO_train2014_{validation_name}.jpg'))
112             validation_name += 1
113
114 def gen_3_by_5_tables(type): # type = 'training' or 'validation'
115     headers = ['Dataset']
116     headers.extend(os.listdir(f'output_dataset/single_instance_dataset'))
117     count = []
118     for dataset_type in os.listdir('output_dataset/'):
119         temp = [dataset_type]
120         for cat in os.listdir(f'output_dataset/{dataset_type}'):
121             files = [f for f in os.listdir(f'output_dataset/{dataset_type}/{cat}/{type}')]
122             temp.append(len(files))
123         count.append(temp)
124     print(tabulate(count, headers=headers, tablefmt='latex'))
125     return
126
127 def dataset_visualization(type):
128     for dataset_type in os.listdir(output_dir_suitable):
129         fig, axes = plt.subplots(5,3)
130         for row, cat in enumerate(os.listdir(os.path.join(output_dir_suitable,
131             dataset_type))):
132             files = [f for f in os.listdir(os.path.join(output_dir_suitable,
133                 dataset_type, cat, type))]
134             for col in range(3):
135                 img = Image.open(os.path.join(output_dir_suitable, dataset_type, cat,
136                     type, files[col]))
137                 axes[row][col].imshow(img)
138                 axes[row][col].axis('off')
139                 fig.text(0.08, 0.9 - (row + 0.5) / 6, cat, ha='center', va='center',
140                     fontsize=12, rotation=0)
141             plt.suptitle(f'{type}:_{dataset_type}')
142             plt.savefig(f'{type}_{dataset_type}.jpg')
143             plt.close(fig)
144     return
145
146 if __name__ == '__main__':
147     if sys.argv[1] == 'gen_data_all':
148         extract_imgs()
149     elif sys.argv[1] == 'check_suitable_dataset':
150         check_suitable_dataset()
151         # From this I selected the following classes: ['airplane', 'bird', 'clock', '
152             giraffe', 'zebra']
153     elif sys.argv[1] == 'gen_suitable_dataset':
154         suitable_cat = ['airplane', 'bird', 'clock', 'giraffe', 'zebra']
155         gen_suitable_dataset(suitable_cat)
156     elif sys.argv[1] == 'gen_table':
157         gen_3_by_5_tables('training')
158         gen_3_by_5_tables('validation')
159     elif sys.argv[1] == 'visualize_data':
160         dataset_visualization('training')
161         dataset_visualization('validation')

```

To create our custom datasets, I first split all the images in each class into the three categories: single_instance_dataset, multi_instance_same_dataset, and multi_instance_different_dataset. This split dataset is saved in a directory called **output_dataset.all**. This first step is executed using the following terminal command: **python GenerateDataset.py gen_data.all**.

After generating the split dataset, the number of images in each of the classes within each category are checked using the terminal command: **python GenerateDataset.py check_suitable_dataset**. After analyzing the output, I chose the following classes to constitute my finally dataset: ['airplane', 'bird', 'clock', 'giraffe', 'zebra'].

To generate the final dataset which will be used for training the CNN in later sections, the following terminal command is used: **python GenerateDataset.py gen_suitable_dataset**. This command calls

the **gen_suitable_dataset()** function, which generates the final dataset within the **output_dataset** directory. If each class chosen contains less than 500 images corresponding to the three dataset types, transforms are applied to original images to generate additional images so that there are 500 images (400 for training and 100 for validation). If there are more than 500 images, only 500 are used.

To check that the number of images generated within the dataset are correct, we run the following terminal command: **python GenerateDataset.py gen_table**. This gives us tables 1 and 2.

Table 1: Number of Samples in Training Dataset

Dataset	airplane	bird	giraffe	clock	zebra
multi_instance_different_dataset	400	400	400	400	400
single_instance_dataset	400	400	400	400	400
multi_instance_same_dataset	400	400	400	400	400

Table 2: Number of Samples in Validation Dataset

Dataset	airplane	bird	giraffe	clock	zebra
multi_instance_different_dataset	100	100	100	100	101
single_instance_dataset	100	100	100	100	101
multi_instance_same_dataset	101	101	100	100	100

For visualizing the images within the dataset, we run the following terminal command: **python GenerateDataset.py visualize_data**.

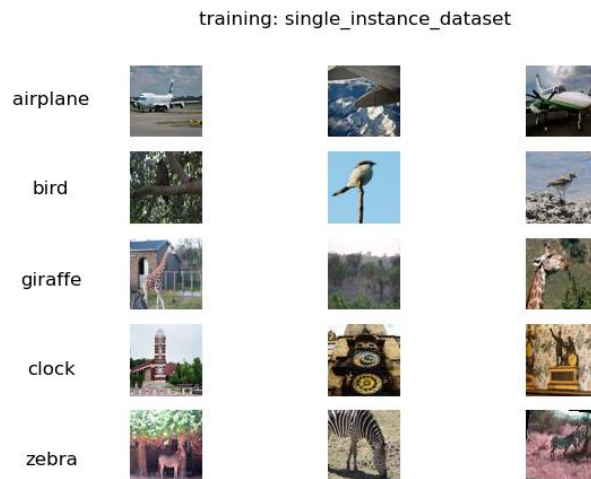


Figure 1: Single Instance Training Dataset

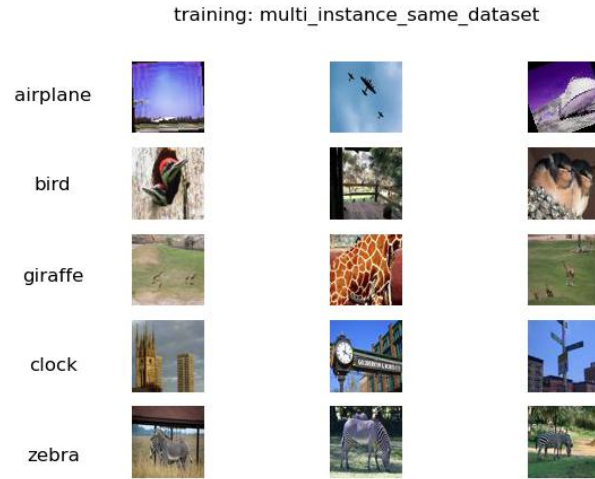


Figure 2: Multi-Instance Same Training Dataset

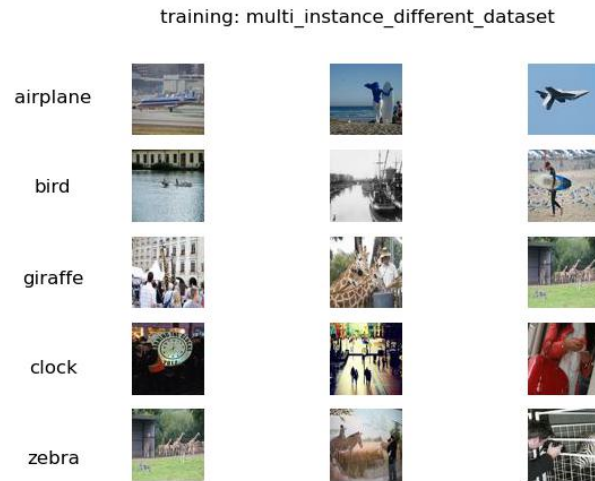


Figure 3: Multi-Instance Different Training Dataset

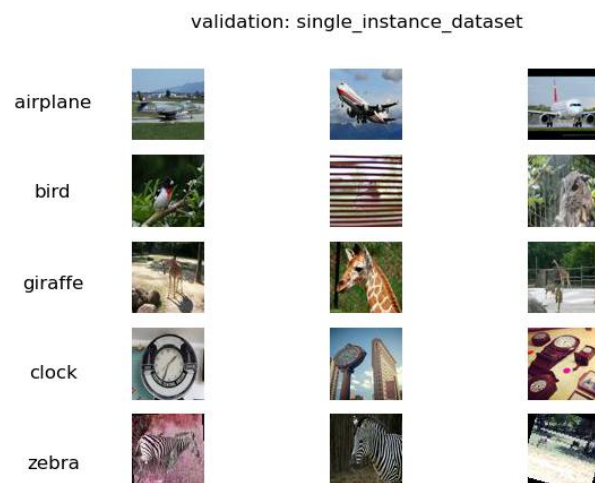


Figure 4: Single Instance Validation Dataset

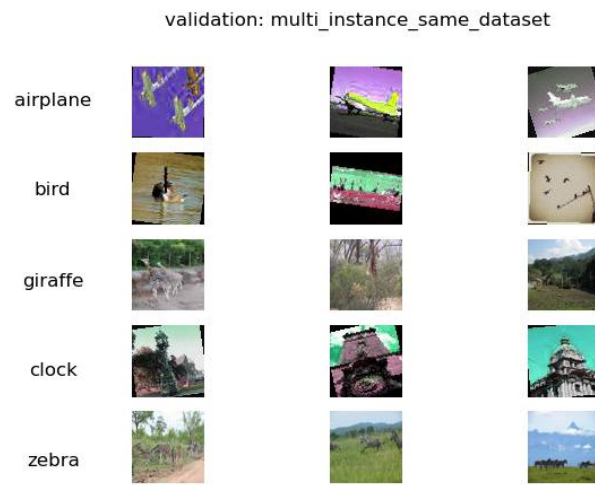


Figure 5: Multi-Instance Same Validation Dataset

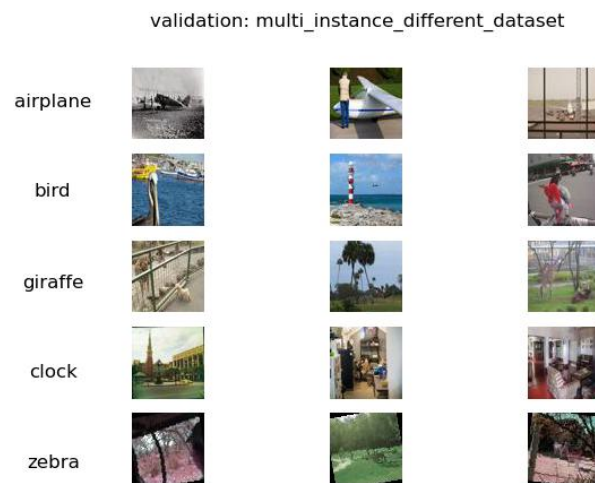


Figure 6: Multi-Instance Different Validation Dataset

2.2 Image Classification using CNNs: HW4Net

2.2.1 Code

In this section, we will be implementing a CNN model with the following architecture:

```
1 class HW4Net(nn.Module):
2     def __init__(self):
3         super(HW4Net, self).__init__()
4         self.conv1 = nn.Conv2d(3, 16, 3)
5         self.pool = nn.MaxPool2d(2, 2)
6         self.conv2 = nn.Conv2d(16, 32, 3)
7         # fill XXXX and XX
8         self.fc1 = nn.Linear(XXXX, 64)
9         self.fc2 = nn.Linear(64, XX)
10
11     def forward(self, x):
12         x = self.pool(F.relu(self.conv1(x)))
13         x = self.pool(F.relu(self.conv2(x)))
14         x = x.view(x.shape[0], -1) # Flatten the tensor before passing it to the fully
15                                     # connected layers
16         x = F.relu(self.fc1(x))
17         x = self.fc2(x)
18         return x
```

In the code snippet above, we must calculate the values for XXXX and XX. To do so, we must calculate the size of the output after each layer:

- Input Size / Image Size: $3 \times 64 \times 64 = C \times N \times N$
- First Convolution: **self.conv1 = nn.Conv2d(3, 16, 3)**
 - (Input Channels, Output Channels, Kernel Size) = (3, 16, 3)
 - Stride = 1 (default)
 - Padding = 0 (default)
 - Output = $\frac{N - \text{KernelSize} + 2\text{Padding}}{\text{Stride}} + 1 = \frac{64 - 3 + 2(0)}{1} + 1 = 62$
 - Output Size = $16 \times 62 \times 62$
- First Max Pooling: **self.pool = nn.MaxPool2d(2, 2)**
 - (Kernel Size, Stride) = (2, 2)
 - Padding = 0 (default)
 - output = $\frac{N - \text{KernelSize} + 2\text{Padding}}{\text{Stride}} + 1 = \frac{62 - 2 + 2(0)}{2} + 1 = 31$
 - Output Size = $16 \times 31 \times 31$
- Second Convolution: **self.conv2 = nn.Conv2d(16, 32, 3)**
 - (Input Channels, Output Channels, Kernel Size) = (16, 32, 3)
 - Stride = 1 (default)
 - Padding = 0 (default)
 - Output = $\frac{N - \text{KernelSize} + 2\text{Padding}}{\text{Stride}} + 1 = \frac{31 - 3 + 2(0)}{1} + 1 = 29$
 - Output Size = $32 \times 29 \times 29$
- Second Max Pooling: **self.pool = nn.MaxPool2d(2, 2)**
 - (Kernel Size, Stride) = (2, 2)
 - Padding = 0 (default)
 - output = $\frac{N - \text{KernelSize} + 2\text{Padding}}{\text{Stride}} + 1 = \frac{29 - 2 + 2(0)}{2} + 1 = 14$
 - Output Size = $32 \times 14 \times 14$
- After Flattening: **x = x.view(x.shape[0], -1)**

– Output Size = $32 \times 14 \times 14 = 6272$

From the above calculations, XXXX = 6272. Since we want to classify the data into five classes, we will set XX = 5.

Before training and testing the CNN, a CustomDataloader class is created. The functions **train_network()** and **test_network()** run the training and testing of the CNN. Results are saved using the **run_train_and_val()** function:

```
1 class CustomDataloader(torch.utils.data.Dataset):
2     def __init__(self, root, action):
3         self.categories = [cat for cat in os.listdir(root)]
4         self.root = root
5         self.img_paths = []
6         self.labels = []
7         for i, cat in enumerate(os.listdir(root)):
8             img_dir = os.path.join(self.root, cat, action)
9             for img in os.listdir(img_dir):
10                 self.img_paths.append(os.path.join(img_dir, img))
11                 self.labels.append(i)
12
13     def __len__(self):
14         return len(self.img_paths)
15
16     def __getitem__(self, index):
17         img_path = self.img_paths[index]
18         img = Image.open(img_path)
19         transform = transforms.Compose([
20             transforms.Grayscale(num_output_channels=3), # some images are Grayscale but
21                 we want images to be 3 X 64 X 64
22             transforms.ToTensor(),
23             transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
24         ])
25         transformed_img = transform(img)
26         label = self.labels[index]
27         return (transformed_img, label)
28
29 def train_network(net, dataset):
30     device = torch.device("cpu")
31     net = net.to(device)
32
33     my_dataset = CustomDataloader(root = f'output_dataset/{dataset}', action = 'training')
34
35     my_dataloader = torch.utils.data.DataLoader(my_dataset, batch_size = 4, shuffle =
36         True, num_workers=2)
37
38     criterion = torch.nn.CrossEntropyLoss()
39     optimizer = torch.optim.Adam(net.parameters(), lr=1e-3, betas=(0.9,0.99))
40     epochs = 7
41
42     training_loss = []
43
44     for epoch in range(epochs):
45         running_loss = 0.0
46
47         for i, data in enumerate(my_dataloader):
48             inputs, labels = data
49             inputs = inputs.to(device)
50             labels = labels.to(device)
51
52             optimizer.zero_grad()
53             outputs = net(inputs)
54             loss = criterion(outputs, labels)
55             loss.backward()
56             optimizer.step()
57             running_loss += loss.item()
58             if (i+1) % 100 == 0:
59                 training_loss.append(running_loss)
60                 print('[epoch: %d, batch: %5d] loss: %.3f' % (epoch + 1, i + 1,
61                     running_loss / 100))
62                 running_loss = 0.0
63         return training_loss
64
65 def test_network(net, dataset):
```

```

62     device = torch.device("cpu")
63     net = net.eval()
64     net = net.to(device)
65
66     my_dataset = CustomDataloader(root = f'output_dataset/{dataset}', action = '
        validation')
67     my_dataloader = torch.utils.data.DataLoader(my_dataset, batch_size = 4, shuffle =
        True, num_workers=2)
68
69     true_labels = []
70     predicted_labels = []
71
72     with torch.no_grad():
73         for i, data in enumerate(my_dataloader):
74             inputs, t_labels = data
75             inputs = inputs.to(device)
76             t_labels = t_labels.to(device)
77             outputs = net(inputs)
78             max_val, p_labels = torch.max(outputs.data,1)
79             true_labels.extend(t_labels.tolist())
80             predicted_labels.extend(p_labels.tolist())
81     return true_labels, predicted_labels
82
83 def calculate_accuracy(true_labels, predicted_labels):
84     count = 0
85     for i, true_label in enumerate(true_labels):
86         if true_label == predicted_labels[i]:
87             count += 1
88     return (count/len(true_labels))*100
89
90 def run_train_and_val(net,title):
91     datasets = ['single_instance_dataset', 'multi_instance_same_dataset', '
        multi_instance_different_dataset']
92     accuracies = []
93     training_loss = []
94     for dataset in datasets:
95         training_loss.append(train_network(net, dataset))
96         true_labels, predicted_labels = test_network(net, dataset)
97         accuracy = calculate_accuracy(true_labels, predicted_labels)
98         accuracies.append(accuracy)
99         conf_matrix = confusion_matrix(np.array(true_labels), np.array(predicted_labels)
100         )
101         sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues')
102         plt.xlabel('Predicted_Labels')
103         plt.ylabel('True_Labels')
104         plt.title(f'Confusion_Matrix:_{title}_{dataset}')
105         plt.savefig(f'Confusion_Matrix:_{title}_{dataset}.jpg')
106         plt.close()
107     print(accuracies)
108     plt.figure()
109     for i in range(3):
110         plt.plot(training_loss[i], label = datasets[i])
111         plt.title(f'Training_Loss:_{title}')
112         plt.xlabel('Iterations')
113         plt.ylabel('Training_Loss')
114         plt.legend()
115         plt.savefig(f'Training_Loss_{title}.jpg')
116     return
117
118 net = HW4Net()
119 run_train_and_val(net, 'HW4Net')

```

2.2.2 Results

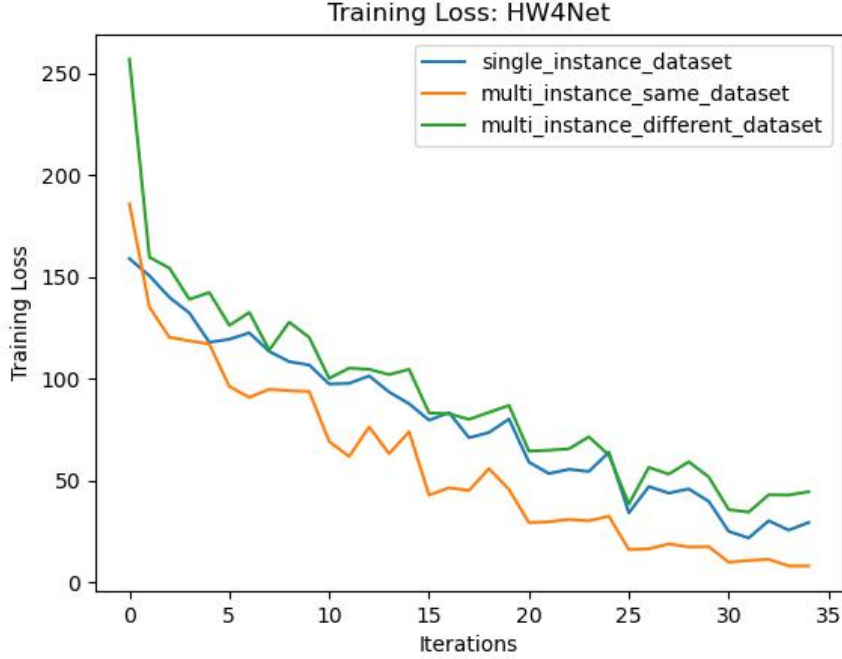


Figure 7: Training Loss for all Datasets

Dataset	Single Instance	Multi Instance Same	Multi Instance Diff
Accuracy	56.09%	53.98%	46.31%

Table 3: Accuracy

Out of all the datasets, the CNN performed best on the single instance dataset, and worst on the different multi-instance dataset. These results make sense because we would expect classification to become more difficult when there are multiple objects in an image, and even more difficult when those multiple objects are different.

The labels on the confusion matrices represent the following classes: $[0, 1, 2, 3, 4] = [\text{'airplane'}, \text{'bird'}, \text{'giraffe'}, \text{'clock'}, \text{'zebra'}]$. From the confusion matrices, we can see that images of zebras were commonly misclassified as giraffes. This is true across all datasets, and especially for the multi-instance same dataset. This makes sense because both zebras and giraffes are animals that walk on four legs. The differentiating factor between zebras and giraffes could be the stripes that zebras have. However, our dataset contains downsized 64×64 images. Thus, whenever there are multiple zebras captured in an image, it is likely that the resolution of those zebras is low and thus, the stripes are not captured in those images. An example of a zebra that was misclassified as a giraffe is shown in figure 11. The zebras in the photo are of such low resolution that their stripes are not discernible.

To improve classification accuracy in the multi-instance different dataset, we can modify the images included in the dataset. For the current multi-instance different dataset, an image is included in the 'airplane' class as long as there is a plane in the image with another type of object. However, we can instead only include the image in the 'airplane' class if the number of appearances of the airplane is larger than the number of appearances of all other objects.

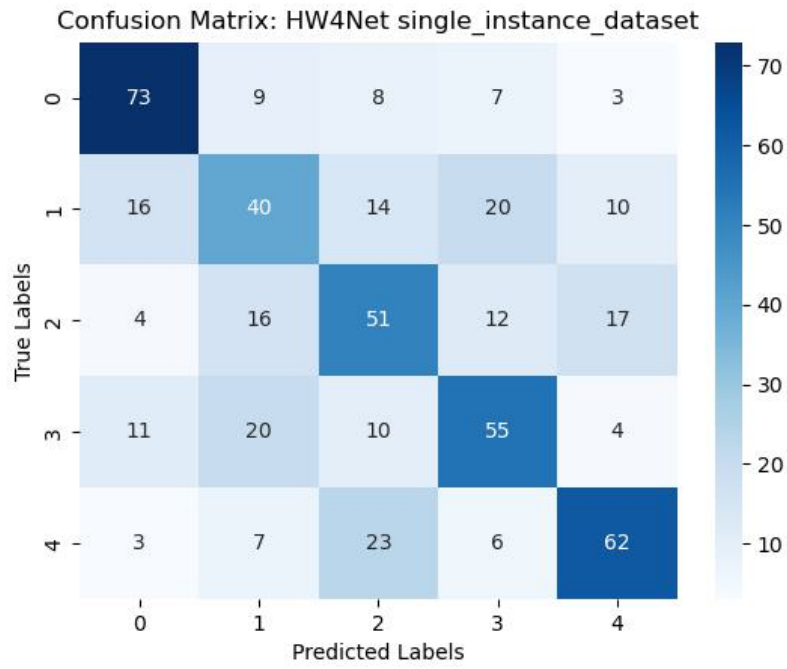


Figure 8: Confusion Matrix for Dataset 1

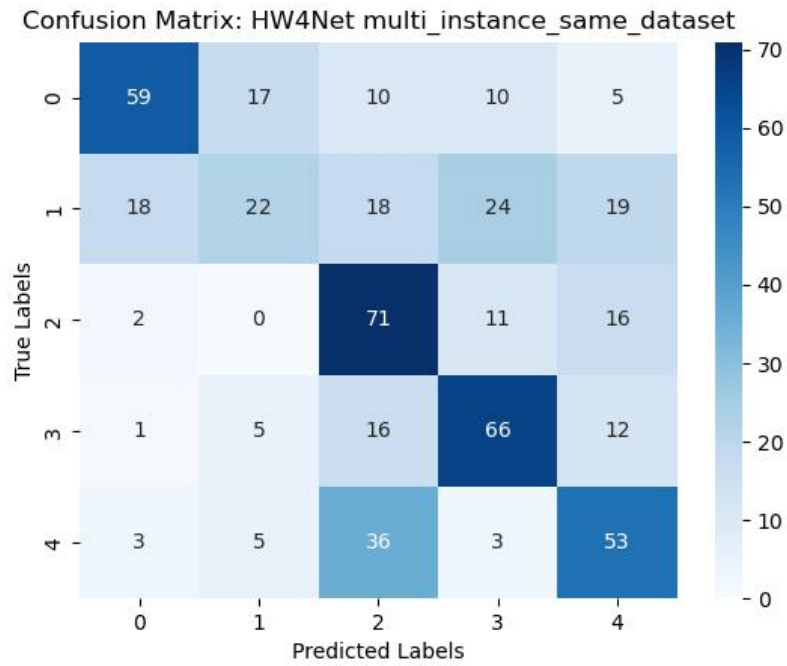


Figure 9: Confusion Matrix for Dataset 2

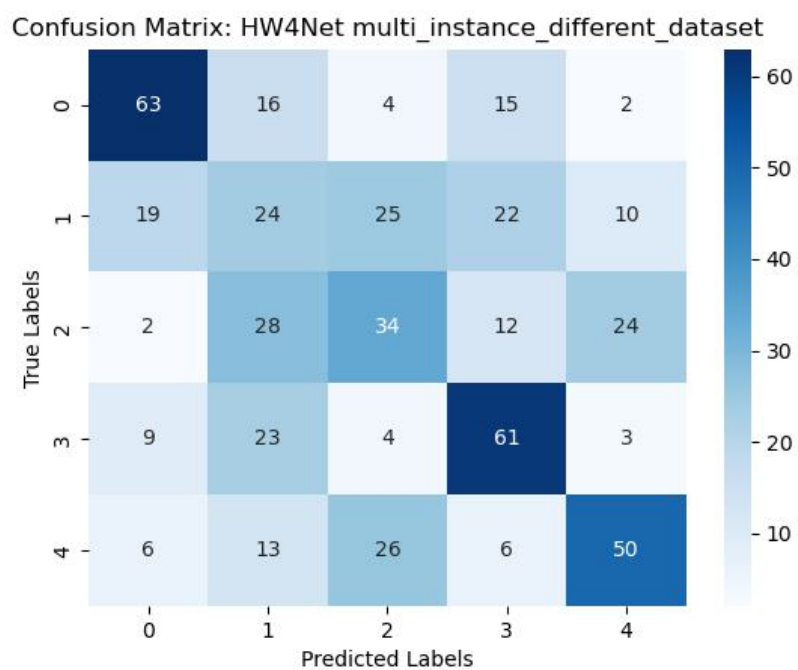


Figure 10: Confusion Matrix for Dataset 3

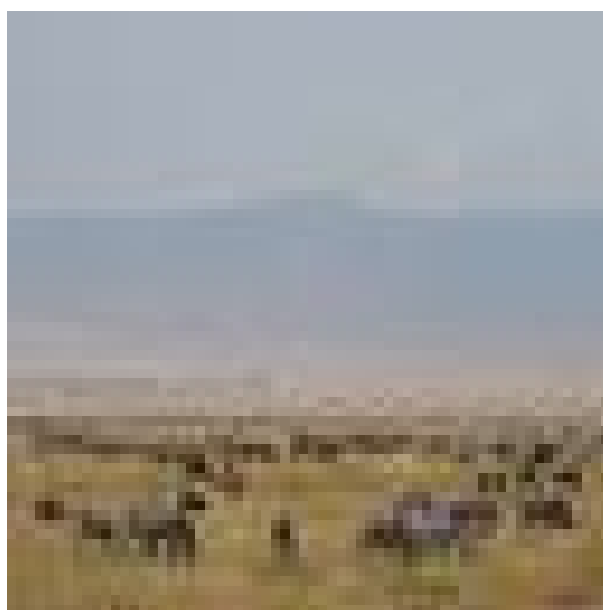


Figure 11: True: Zebra; Predicted: Giraffe

3 BONUS

3.1 Image Classification using CNNs: HW4Net1

3.1.1 Code

In this section, the Baseline TheseCNN model is extended by creating a deeper architecture with 5 convolutional layers. The following code shows the architecture for this extended model:

```
1 class HW4Net1(nn.Module):
2     def __init__(self):
3         super(HW4Net1, self).__init__()
4         self.conv_seqn = nn.Sequential(
5             nn.Conv2d(3, 16, 3, padding=1),
6             nn.ReLU(),
7             nn.Conv2d(16, 32, 3, padding=1),
8             nn.ReLU(),
9             nn.MaxPool2d(2,2),
10
11             nn.Conv2d(32, 64, 3, padding=1),
12             nn.ReLU(),
13             nn.Conv2d(64, 128, 3, padding=1),
14             nn.ReLU(),
15             nn.MaxPool2d(2,2),
16
17             nn.Conv2d(128, 256, 3, padding=1),
18             nn.ReLU(),
19             nn.MaxPool2d(2,2)
20         )
21
22         self.fc_seqn = nn.Sequential(
23             nn.Linear(256*8*8, 128),
24             nn.ReLU(),
25             nn.Linear(128, 5)
26         )
```

The flattened size of $256 \times 8 \times 8$ is calculated as follows:

- First Convolutional Block
 - **Input:** (3, 64, 64)
 - Conv2D ($3 \rightarrow 16, 3 \times 3, \text{padding} = 1$) $\rightarrow$ (16, 64, 64)
 - Conv2D ($16 \rightarrow 32, 3 \times 3, \text{padding} = 1$) $\rightarrow$ (32, 64, 64)
 - MaxPool (2×2) $\rightarrow$ (32, 32, 32)
- Second Convolutional Block
 - **Input:** (3, 64, 64)
 - Conv2D ($3 \rightarrow 16, 3 \times 3, \text{padding} = 1$) $\rightarrow$ (16, 64, 64)
 - Conv2D ($16 \rightarrow 32, 3 \times 3, \text{padding} = 1$) $\rightarrow$ (32, 64, 64)
 - MaxPool (2×2) $\rightarrow$ (32, 32, 32)
- Third Convolutional Block
 - Conv2D ($128 \rightarrow 256, 3 \times 3, \text{padding} = 1$) $\rightarrow$ (256, 16, 16)
 - MaxPool (2×2) $\rightarrow$ (256, 8, 8)

3.1.2 Results

Dataset	Single Instance	Multi Instance Same	Multi Instance Diff
Accuracy	59.28%	53.58%	44.71%

Table 4: Accuracy

When additional convolutional layers were added to the network, the accuracy for the single instance dataset improved by 3.18%. For the multi-instance datasets, the accuracy for the same instance dataset remained almost the same while the accuracy for the difference instance dataset decreased by 1.6%.

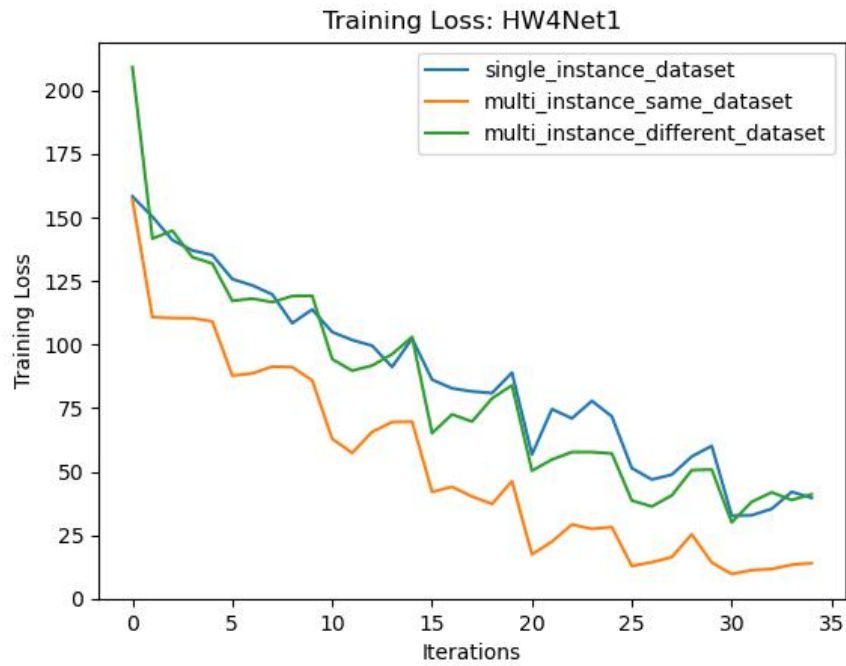


Figure 12: Training Loss for all Datasets

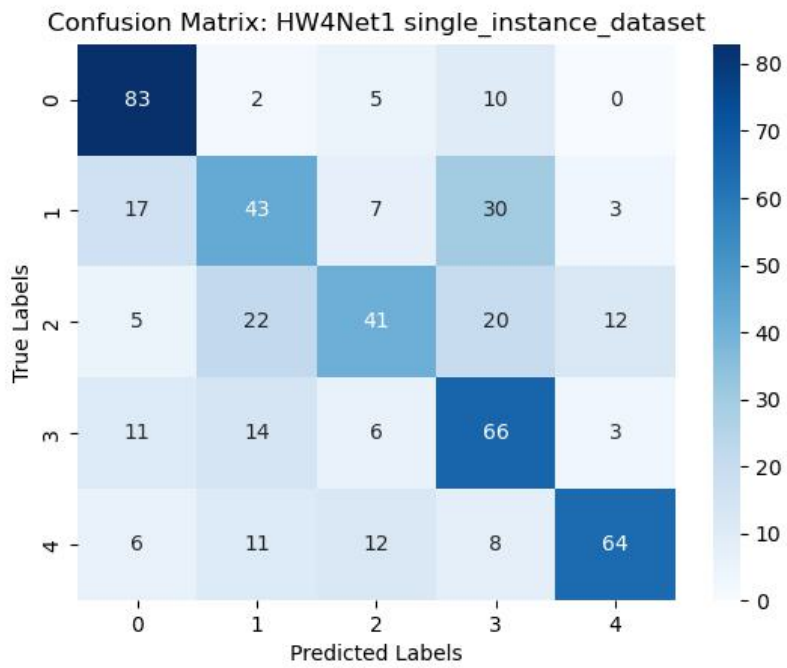


Figure 13: Confusion Matrix for Dataset 1



Figure 14: Confusion Matrix for Dataset 2

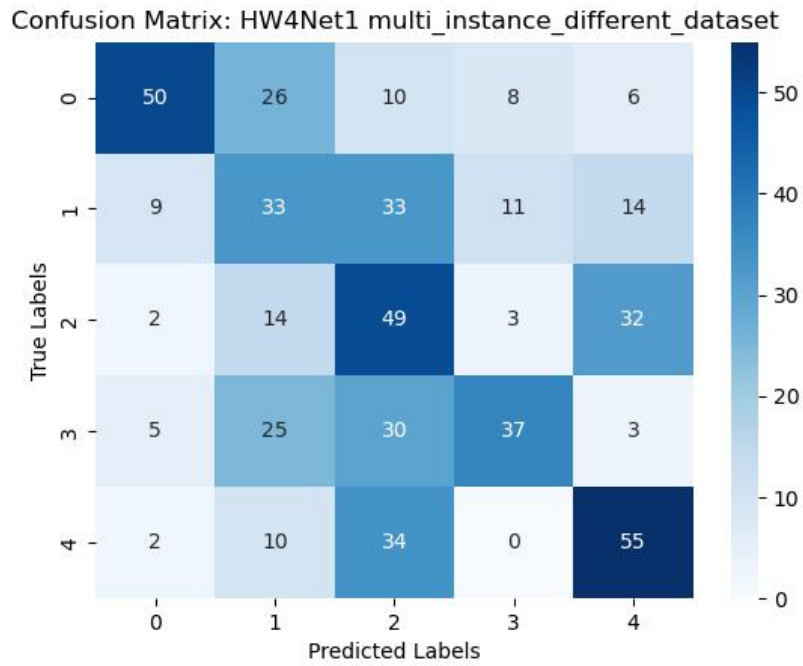


Figure 15: Confusion Matrix for Dataset 3

3.2 Image Classification using CNNs: HW4Net2

3.2.1 Code

```
1 class HW4Net2(nn.Module):
2     def __init__(self):
3         super(HW4Net2, self).__init__()
4         self.conv_seqn = nn.Sequential(
5             nn.Conv2d(3, 16, 3, padding=1),
6             nn.ReLU(),
7             nn.Conv2d(16, 32, 3, padding=1),
8             nn.ReLU(),
9             nn.MaxPool2d(2,2),
10
11             nn.Conv2d(32, 64, 3, padding=1),
12             nn.ReLU(),
13             nn.Conv2d(64, 128, 3, padding=1),
14             nn.ReLU(),
15             nn.MaxPool2d(2,2),
16
17             nn.Conv2d(128, 256, 3, padding=1),
18             nn.ReLU(),
19             nn.Conv2d(256, 512, 3, padding=1),
20             nn.ReLU(),
21             nn.MaxPool2d(2,2),
22
23             nn.Conv2d(512, 512, 3, padding=1),
24             nn.ReLU(),
25             nn.Conv2d(512, 512, 3, padding=1),
26             nn.ReLU(),
27             nn.MaxPool2d(2,2),
28
29             nn.Conv2d(512, 512, 3, padding=1),
30             nn.ReLU(),
31             nn.Conv2d(512, 512, 3, padding=1),
32             nn.ReLU(),
33             nn.MaxPool2d(2,2)
34         )
35
36         self.fc_seqn = nn.Sequential(
37             nn.Linear(512 * 2 * 2, 256),
38             nn.ReLU(),
39             nn.Linear(256, 5)
40         )
41
42     def forward(self, x):
43         x = self.conv_seqn(x)
44         x = x.view(x.shape[0], -1) # Flatten the tensor before passing it to the fully
45                                     # connected layers
46         x = self.fc_seqn(x)
47         return x
```

3.2.2 Results

When ten convolutional layers are used, the model seems to completely fail at classification for all three datasets. Training loss does not decrease at all, which means that the model was not able to learn any meaningful features. The outcome is that the model predicts the same class for every validation image, as shown by the following confusion matrices.

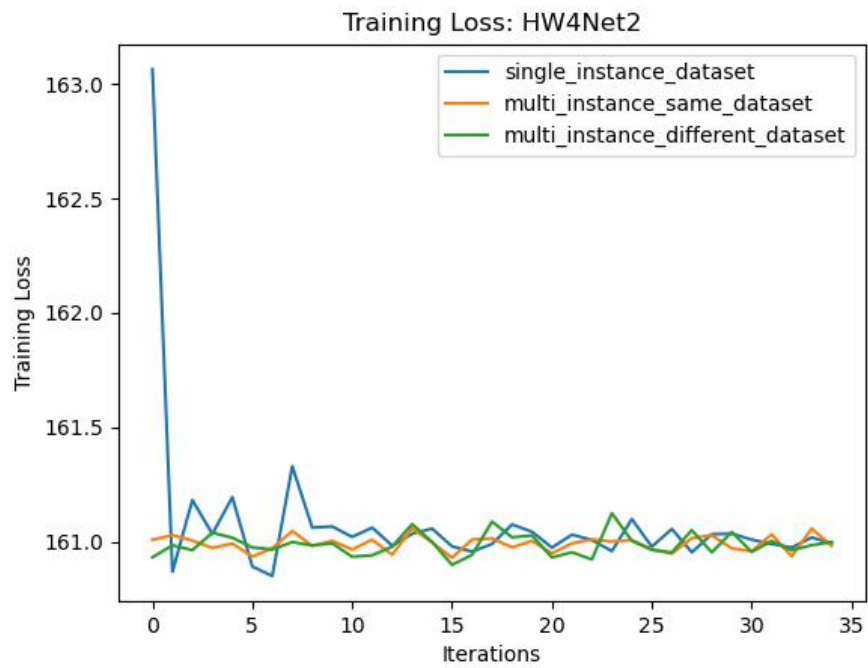


Figure 16: Training Loss for all Datasets

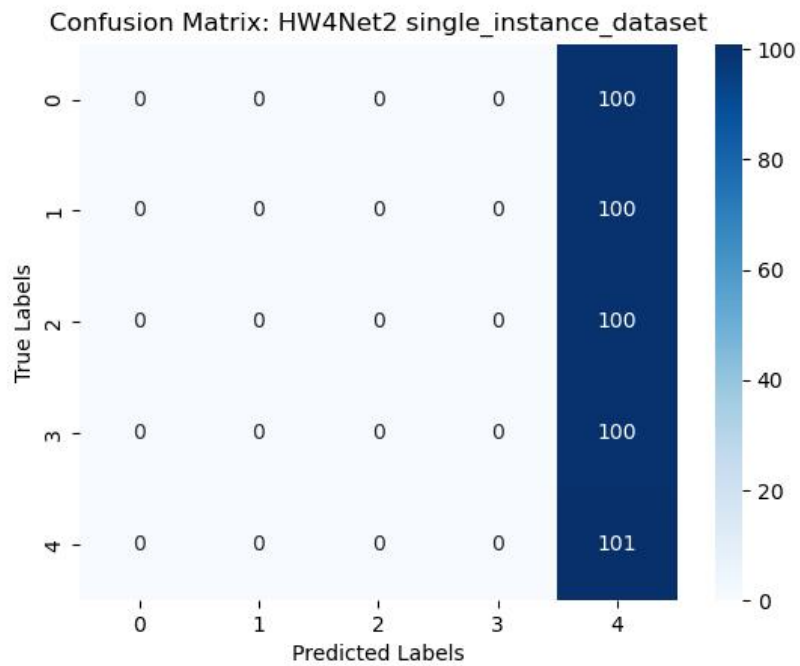


Figure 17: Confusion Matrix for Dataset 1



Figure 18: Confusion Matrix for Dataset 2

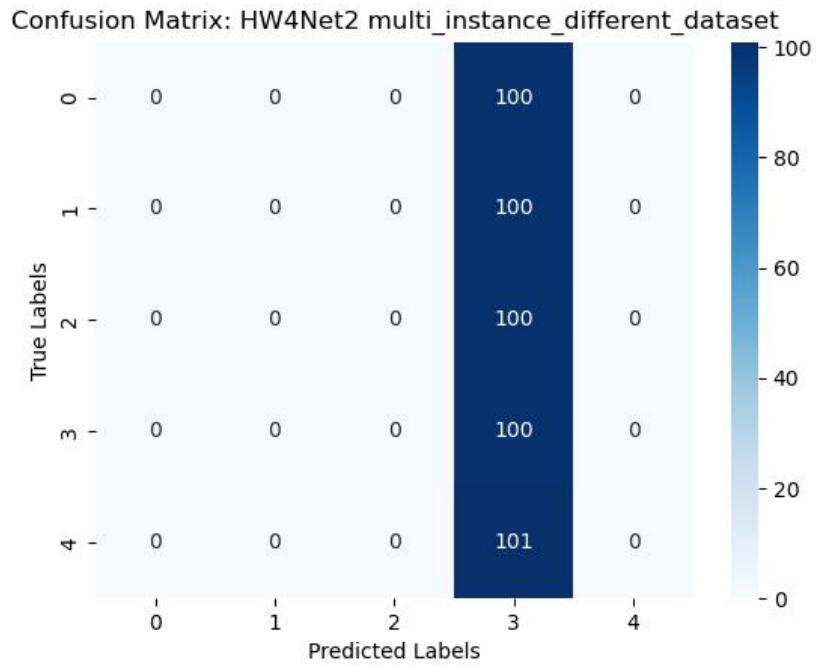


Figure 19: Confusion Matrix for Dataset 3

4 CODE: Dataset Generation

```
1 from pycocotools.coco import COCO
2 import os
3 from PIL import Image
4 import sys
5 import matplotlib.pyplot as plt
6 from torchvision import transforms
7 from tabulate import tabulate
8
9 # Set COCO dataset paths
10 img_dir = 'train2014'
11 ann_file = 'annotations_trainval2014/annotations/instances_train2014.json'
12 output_dir_all = 'output_datasets_all'
13 output_dir_suitable = 'output_dataset'
14
15 # Load COCO dataset
16 coco = COCO(ann_file)
17
18 # Ensure output directories exist
19 os.makedirs(output_dir_all, exist_ok=True)
20 os.makedirs(output_dir_suitable, exist_ok=True)
21
22 def extract_imgs():
23     cat_ids = coco.getCatIds()
24     categories = coco.loadCats(cat_ids)
25     cat_names = [cat['name'] for cat in categories]
26     for cat_name in cat_names:
27         cat_ids = coco.getCatIds(catNms=cat_name)
28         img_ids = coco.getImgIds(catIds=cat_ids)
29
30         for img_id in img_ids:
31             img_info = coco.loadImgs(img_id)[0]
32             ann_ids = coco.getAnnIds(imgIds=img_id, iscrowd=False)
33             anns = coco.loadAnns(ann_ids)
34
35             obj_counts = {} # Initialize dictionary to keep track of object categories
36                             # and count
37
38             for ann in anns:
39                 obj_category = coco.loadCats(ann['category_id'])[0]['name']
40                 obj_counts[obj_category] = obj_counts.get(obj_category, 0) + 1
41
42             if len(obj_counts) >= 2: # if multiple object categories are present
43                 save_image(img_info, cat_name, 'multi_instance_different_dataset')
44             else:
45                 if obj_counts[obj_category] == 1:
46                     save_image(img_info, cat_name, 'single_instance_dataset')
47                 else:
48                     save_image(img_info, cat_name, 'multi_instance_same_dataset')
49
50 def save_image(img_info, category_name, dataset_type):
51     img_path = os.path.join(img_dir, img_info['file_name'])
52     save_dir = os.path.join(output_dir_all, dataset_type, category_name)
53     os.makedirs(save_dir, exist_ok=True)
54
55     # Load and resize image
56     img = Image.open(img_path).resize((64, 64))
57     img.save(os.path.join(save_dir, img_info['file_name']))
58
59 def check_suitable_dataset():
60     for dataset_type in os.listdir(output_dir_all):
61         print(f'Suitable for {dataset_type}')
62         for cat in os.listdir(os.path.join(output_dir_all, dataset_type)):
63             files = [f for f in os.listdir(os.path.join(output_dir_all, dataset_type,
64                                                             cat))]
65             if len(files) >= 500:
66                 print(f'{cat}: {len(files)} (Without augmentation)')
67             elif len(files) >= 350:
68                 print(f'{cat}: {len(files)} (With < 150 augmentations)')
69             elif len(files) >= 300:
70                 print(f'{cat}: {len(files)} (With 150 - 200 augmentations)')
```

```

70         elif len(files) >= 250:
71             print(f'{cat}:\_{len(files)}\_(With\_{200-250}\_augmentations)')
72     return
73
74 def gen_suitable_dataset(suitable_cat):
75     for dataset_type in os.listdir(output_dir_all):
76         for cat in suitable_cat:
77             files = [f for f in os.listdir(os.path.join(output_dir_all, dataset_type,
78                 cat))]
79             num_training = min(400, int(0.8*len(files)))
80             num_total = min(500, len(files))
81             save_dir_train = os.path.join(output_dir_suitable, dataset_type, cat, '
                training')
82             save_dir_validation = os.path.join(output_dir_suitable, dataset_type, cat, '
                validation')
83             for file in files[:num_training]:
84                 img = Image.open(os.path.join(output_dir_all, dataset_type, cat, file))
85                 os.makedirs(save_dir_train, exist_ok=True)
86                 img.save(os.path.join(save_dir_train, file))
87             for file in files[num_training:num_total]:
88                 img = Image.open(os.path.join(output_dir_all, dataset_type, cat, file))
89                 os.makedirs(save_dir_validation, exist_ok=True)
90                 img.save(os.path.join(save_dir_validation, file))
91
92             if num_total < 500:
93                 transform_custom = transforms.Compose([
94                     transforms.RandomHorizontalFlip(),
95                     transforms.RandomRotation(20),
96                     transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2,
97                         hue=0.2),
98                 ])
99                 train_name = int(files[num_training-1][15:27]) + 1
100                 validation_name = int(files[-1][15:27]) + 1
101                 for i in range(400-int(0.8*(num_total))):
102                     img = Image.open(os.path.join(output_dir_all, dataset_type, cat,
103                         files[i%num_total]))
104                     img_transformed = transform_custom(img)
105                     img_transformed.save(os.path.join(save_dir_train, f'COCO_train2014_{
106                         train_name}.jpg'))
107                     train_name += 1
108                 for i in range(100-int(0.2*num_total)):
109                     img = Image.open(os.path.join(output_dir_all, dataset_type, cat,
110                         files[len(files)-(i%(num_total-num_training))-1]))
111                     img_transformed = transform_custom(img)
112                     img_transformed.save(os.path.join(save_dir_validation, f'
113                         COCO_train2014_{validation_name}.jpg'))
114                     validation_name += 1
115
116 def gen_3_by_5_tables(type): # type = 'training' or 'validation'
117     headers = ['Dataset']
118     headers.extend(os.listdir(f'output_dataset/single_instance_dataset'))
119     count = []
120     for dataset_type in os.listdir('output_dataset/'):
121         temp = [dataset_type]
122         for cat in os.listdir(f'output_dataset/{dataset_type}'):
123             files = [f for f in os.listdir(f'output_dataset/{dataset_type}/{cat}/{type}')]
124             temp.append(len(files))
125         count.append(temp)
126     print(tabulate(count, headers=headers, tablefmt='latex'))
127     return
128
129 def dataset_visualization(type):
130     for dataset_type in os.listdir(output_dir_suitable):
131         fig, axes = plt.subplots(5,3)
132         for row, cat in enumerate(os.listdir(os.path.join(output_dir_suitable,
133             dataset_type))):
134             files = [f for f in os.listdir(os.path.join(output_dir_suitable,
135                 dataset_type, cat, type))]
136             for col in range(3):
137                 img = Image.open(os.path.join(output_dir_suitable, dataset_type, cat,
138                     type, files[col]))
139                 axes[row][col].imshow(img)

```

```

131         axes[row][col].axis('off')
132         fig.text(0.08, 0.9 - (row + 0.5) / 6, cat, ha='center', va='center',
133                 fontsize=12, rotation=0)
133     plt.suptitle(f'[{type}]:_{dataset_type}')
134     plt.savefig(f'[{type}]_{dataset_type}.jpg')
135     plt.close(fig)
136     return
137
138 if __name__ == '__main__':
139     if sys.argv[1] == 'gen_data_all':
140         extract_imgs()
141     elif sys.argv[1] == 'check_suitable_dataset':
142         check_suitable_dataset()
143         # From this I selected the following classes: ['airplane', 'bird', 'clock', '
144         # giraffe', 'zebra']
145     elif sys.argv[1] == 'gen_suitable_dataset':
146         suitable_cat = ['airplane', 'bird', 'clock', 'giraffe', 'zebra']
147         gen_suitable_dataset(suitable_cat)
148     elif sys.argv[1] == 'gen_table':
149         gen_3_by_5_tables('training')
150         gen_3_by_5_tables('validation')
151     elif sys.argv[1] == 'visualize_data':
152         dataset_visualization('training')
153         dataset_visualization('validation')

```

5 CODE: CNN training and validation

```

1  import torch.nn as nn
2  import torch.nn.functional as F
3  import os
4  from PIL import Image
5  import torch
6  import torch.utils.data.dataloader
7  import matplotlib.pyplot as plt
8  from torchvision import transforms
9  from sklearn.metrics import confusion_matrix
10 import seaborn as sns
11 import numpy as np
12 import sys
13
14 class HW4Net(nn.Module):
15     def __init__(self):
16         super(HW4Net, self).__init__()
17         self.conv1 = nn.Conv2d(3, 16, 3)
18         self.pool = nn.MaxPool2d(2, 2)
19         self.conv2 = nn.Conv2d(16, 32, 3)
20
21         self.fc1 = nn.Linear(6272, 64)
22         self.fc2 = nn.Linear(64, 5)
23
24     def forward(self, x):
25         x = self.pool(F.relu(self.conv1(x)))
26         x = self.pool(F.relu(self.conv2(x)))
27         x = x.view(x.shape[0], -1) # Flatten the tensor before passing it to the fully
28         # connected layers
29         x = F.relu(self.fc1(x))
30         x = self.fc2(x)
31         return x
32
33 class HW4Net1(nn.Module):
34     def __init__(self):
35         super(HW4Net1, self).__init__()
36         self.conv_seqn = nn.Sequential(
37             nn.Conv2d(3, 16, 3, padding=1),
38             nn.ReLU(),
39             nn.Conv2d(16, 32, 3, padding=1),
40             nn.ReLU(),
41             nn.MaxPool2d(2,2),
42             nn.Conv2d(32, 64, 3, padding=1),

```

```

43         nn.ReLU(),
44         nn.Conv2d(64, 128, 3, padding=1),
45         nn.ReLU(),
46         nn.MaxPool2d(2,2),
47
48         nn.Conv2d(128, 256, 3, padding=1),
49         nn.ReLU(),
50         nn.MaxPool2d(2,2)
51     )
52
53     self.fc_seqn = nn.Sequential(
54         nn.Linear(256*8*8, 128),
55         nn.ReLU(),
56         nn.Linear(128, 5)
57     )
58
59     def forward(self, x):
60         x = self.conv_seqn(x)
61         x = x.view(x.shape[0], -1) # Flatten the tensor before passing it to the fully
            connected layers
62         x = self.fc_seqn(x)
63         return x
64
65 class HW4Net2(nn.Module):
66     def __init__(self):
67         super(HW4Net2, self).__init__()
68         self.conv_seqn = nn.Sequential(
69             nn.Conv2d(3, 16, 3, padding=1),
70             nn.ReLU(),
71             nn.Conv2d(16, 32, 3, padding=1),
72             nn.ReLU(),
73             nn.MaxPool2d(2,2),
74
75             nn.Conv2d(32, 64, 3, padding=1),
76             nn.ReLU(),
77             nn.Conv2d(64, 128, 3, padding=1),
78             nn.ReLU(),
79             nn.MaxPool2d(2,2),
80
81             nn.Conv2d(128, 256, 3, padding=1),
82             nn.ReLU(),
83             nn.Conv2d(256, 512, 3, padding=1),
84             nn.ReLU(),
85             nn.MaxPool2d(2,2),
86
87             nn.Conv2d(512, 512, 3, padding=1),
88             nn.ReLU(),
89             nn.Conv2d(512, 512, 3, padding=1),
90             nn.ReLU(),
91             nn.MaxPool2d(2,2),
92
93             nn.Conv2d(512, 512, 3, padding=1),
94             nn.ReLU(),
95             nn.Conv2d(512, 512, 3, padding=1),
96             nn.ReLU(),
97             nn.MaxPool2d(2,2)
98         )
99
100     self.fc_seqn = nn.Sequential(
101         nn.Linear(512 * 2 * 2, 256),
102         nn.ReLU(),
103         nn.Linear(256, 5)
104     )
105
106     def forward(self, x):
107         x = self.conv_seqn(x)
108         x = x.view(x.shape[0], -1) # Flatten the tensor before passing it to the fully
            connected layers
109         x = self.fc_seqn(x)
110         return x
111
112
113 class CustomDataLoader(torch.utils.data.Dataset):

```

```

114     def __init__(self, root, action):
115         self.categories = [cat for cat in os.listdir(root)]
116         self.root = root
117         self.img_paths = []
118         self.labels = []
119         for i, cat in enumerate(os.listdir(root)):
120             img_dir = os.path.join(self.root, cat, action)
121             for img in os.listdir(img_dir):
122                 self.img_paths.append(os.path.join(img_dir, img))
123                 self.labels.append(i)
124
125     def __len__(self):
126         return len(self.img_paths)
127
128     def __getitem__(self, index):
129         img_path = self.img_paths[index]
130         self.img = Image.open(img_path)
131         transform = transforms.Compose([
132             transforms.Grayscale(num_output_channels=3), # some images are Grayscale but
133                 we want images to be 3 X 64 X 64
134             transforms.ToTensor(),
135             transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
136         ])
137         transformed_img = transform(self.img)
138         label = self.labels[index]
139         return (transformed_img, label)
140
141 def train_network(net, dataset):
142     device = torch.device("cpu")
143     net = net.to(device)
144
145     my_dataset = CustomDataloader(root = f'output_dataset/{dataset}', action = 'training')
146     my_dataloader = torch.utils.data.DataLoader(my_dataset, batch_size = 4, shuffle =
        True, num_workers=2)
147
148     criterion = torch.nn.CrossEntropyLoss()
149     optimizer = torch.optim.Adam(net.parameters(), lr=1e-3, betas=(0.9,0.99))
150     epochs = 7
151
152     training_loss = []
153
154     for epoch in range(epochs):
155         running_loss = 0.0
156
157         for i, data in enumerate(my_dataloader):
158             inputs, labels = data
159             inputs = inputs.to(device)
160             labels = labels.to(device)
161
162             optimizer.zero_grad()
163             outputs = net(inputs)
164             loss = criterion(outputs, labels)
165             loss.backward()
166             optimizer.step()
167             running_loss += loss.item()
168             if (i+1) % 100 == 0:
169                 training_loss.append(running_loss)
170                 print('[epoch: %d, batch: %5d] loss: %.3f' % (epoch + 1, i + 1,
171                     running_loss / 100))
172                 running_loss = 0.0
173         return training_loss
174
175 def test_network(net, dataset):
176     device = torch.device("cpu")
177     net = net.eval()
178     net = net.to(device)
179
180     my_dataset = CustomDataloader(root = f'output_dataset/{dataset}', action = '
        validation')
181     my_dataloader = torch.utils.data.DataLoader(my_dataset, batch_size = 4, shuffle =
        True, num_workers=2)

```

```

181
182 true_labels = []
183 predicted_labels = []
184
185 with torch.no_grad():
186     for i, data in enumerate(my_dataloader):
187         inputs, t_labels = data
188         inputs = inputs.to(device)
189         t_labels = t_labels.to(device)
190         outputs = net(inputs)
191         max_val, p_labels = torch.max(outputs.data, 1)
192         true_labels.extend(t_labels.tolist())
193         predicted_labels.extend(p_labels.tolist())
194     return true_labels, predicted_labels
195
196 def calculate_accuracy(true_labels, predicted_labels):
197     count = 0
198     wrong_index = []
199     for i, true_label in enumerate(true_labels):
200         if true_label == predicted_labels[i]:
201             count += 1
202     return (count/len(true_labels))*100
203
204 def run_train_and_val(net, title):
205     datasets = ['single_instance_dataset', 'multi_instance_same_dataset', '
206                 multi_instance_different_dataset']
207     accuracies = []
208     training_loss = []
209     for dataset in datasets:
210         training_loss.append(train_network(net, dataset))
211         true_labels, predicted_labels = test_network(net, dataset)
212         accuracy = calculate_accuracy(true_labels, predicted_labels)
213         accuracies.append(accuracy)
214         conf_matrix = confusion_matrix(np.array(true_labels), np.array(predicted_labels))
215         sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues')
216         plt.xlabel('Predicted_Labels')
217         plt.ylabel('True_Labels')
218         plt.title(f'Confusion_Matrix:_{title}_{dataset}')
219         plt.savefig(f'Confusion_Matrix:_{title}_{dataset}.jpg')
220         plt.close()
221     print(accuracies)
222     plt.figure()
223     for i in range(3):
224         plt.plot(training_loss[i], label = datasets[i])
225         plt.title(f'Training_Loss:_{title}')
226         plt.xlabel('Iterations')
227         plt.ylabel('Training_Loss')
228         plt.legend()
229         plt.savefig(f'Training_Loss:_{title}.jpg')
230     return
231
232 if __name__ == '__main__':
233     if sys.argv[1] == '0':
234         net = HW4Net()
235         run_train_and_val(net, 'HW4Net')
236     elif sys.argv[1] == '1':
237         net = HW4Net1()
238         run_train_and_val(net, 'HW4Net1')
239     elif sys.argv[1] == '2':
240         net = HW4Net2()
241         run_train_and_val(net, 'HW4Net2')

```
